

Predicting TTC Subway Delays from Weather Data

JSC370 Final Project Report

Xiran Yin

2026-04-26

1 Introduction

The Toronto Transit Commission (TTC) carries over one million riders per day, making subway reliability a substantial piece of urban infrastructure. While many causes of delay — equipment failures, passenger incidents, security investigations — are random from a forecasting perspective, **weather** is one of the few systematic, externally observable, and forecastable variables that plausibly drives day-to-day delay patterns. Cold snaps freeze track switches; heavy snow buries third rails; extreme heat causes rail steel to expand. This project asks whether daily Toronto weather predicts TTC delays, by integrating 12 years of meteorological observations (2014–2025) with the City of Toronto’s complete public record of subway delays.

1.1 Central Research Question

Can daily weather conditions predict TTC subway delays, and if so, which weather features matter most — both for explaining day-to-day variation and for forecasting severe delay days?

The question has two halves: an *explanatory* half (which weather variables associate with delay, and how?) and a *predictive* half (can that association be turned into a forecasting tool?). Each maps to a formal hypothesis with an explicit null.

H — Explanatory. Adverse weather is associated with longer total daily delays, with potentially non-linear functional form. *Null:* the partial slopes on precipitation and snowfall are zero, and the smooths are flat, after controlling for month and day-of-week. *Alternative:* slopes are positive; smooths exhibit non-linear structure (e.g., U-shaped temperature). H is tested with OLS and a Generalized Additive Model (GAM).

H — Predictive. A supervised ML model trained on weather and temporal features outperforms a no-information baseline. *Null:* out-of-sample $R^2 \approx 0$ and ROC-AUC ≈ 0.5 — i.e., no better than predicting the historical mean or majority class. *Alternative:* $R^2 > 0$ and ROC-AUC > 0.5 on a chronologically held-out test set. H is tested by training Linear/Logistic Regression, Random Forest, and XGBoost on 2014–2023 data and evaluating on 2024–2025.

GitHub repo: <https://github.com/xiran-yin/JSC370-finalproject>.

2 Methods

2.1 Data Sources

Two independent sources are combined: (1) **Weather** — [Open-Meteo Historical Weather API](#), Toronto (43.7064°N, −79.3986°W), daily aggregates 2014–2026; (2) **TTC delays** — City of Toronto Open Data Portal [ttc-subway-delay-data](#), every recorded incident 2014–2025, retrieved via the CKAN API.

2.2 Data Acquisition and Cleaning

Weather acquisition is one HTTPS GET to the Open-Meteo archive. TTC acquisition handles three obstacles: (i) **format dispatching** (mixed CSV/XLSX requires different pandas readers); (ii) **multi-sheet Excel** (2014–2017 files split records across 12 monthly worksheets — `sheet_name=None` is required to avoid silently dropping ~92% of those years); (iii) **inconsistent column names** (“Date” / “Occurrence Date” / “Report Date”; “Min Delay” / “min_delay” / “Minimum Delay”) — handled by substring-matching against known synonyms.

The merged dataset has **4,384 daily observations**. Temporal features (month, day-of-week, weekend) and a three-level **categorical weather severity variable** are derived. Severity uses two fixed breakpoints — `precip_mm 10 or snowfall_cm 5` for “Heavy”; either positive but neither at threshold for “Light”; otherwise “Clear” — corresponding to Environment Canada’s heavy-rain advisory level and a snow depth at which clearing becomes operationally relevant. The binary classification target `is_severe` flags days at or above the 75th percentile of `total_delay_mins`; we additionally model the continuous total directly (regression) and report a 90th-percentile sensitivity check, so conclusions do not depend on a single discretization.

2.3 Statistical and Machine-Learning Methods

Inferential: (i) **OLS multiple regression** of `total_delay_mins` on precipitation, snowfall, gust, mean temperature, weekend, and month FE, as a linear baseline. (ii) **GAM** via `pygam.LinearGAM(s(precip) + s(temp) + f(weekend) + f(month))`, with 20-knot penalized cubic splines and smoothing parameters chosen by generalized cross-validation, to capture non-linearity.

Predictive: Three supervised models fit twice — once for regression (target: `total_delay_mins`) and once for classification (target: `is_severe`). The models are **Linear/Logistic Regression** (baseline), **Random Forest** (`sklearn.ensemble`), and **XGBoost** (`xgboost`). Hyperparameters: RF — `n_estimators=300–400`, `max_depth=8–10`; XGBoost — `n_estimators=400`, `max_depth=6`, `learning_rate=0.05`, `subsample=0.8`. Twelve features are used (`precip_mm`, `snowfall_cm`, `precip_hours`, `precip_intensity`, `temp_max`, `temp_min`, `mean_temp_c`, `temp_range`, `gust_speed`, `is_weekend_int`, `month_num`, `dow_num`). The data are split **chronologically** — train on 2014-01-01 to 2023-12-31, test on 2024-01-01 onward — to prevent the model from learning future information. Regression metrics: RMSE, MAE, R^2 . Classification metrics: accuracy, precision, recall, F1, ROC-AUC.

3 Results

3.1 Descriptive Statistics

Table 1. Descriptive statistics, 2014–2025.

Table 1

	N	Mean	SD	Min	Q1	Median	Q3	Max
Total Delay (min)	4384	144.31	96.88	4	84	123	179	2342
Total Incidents	4384	56.6	16.24	15	45	56	67	144
Mean Temp (°C)	4384	8.97	10.22	-21.3	1	8.8	18.3	31.35
Precipitation (mm)	4384	2.38	4.95	0	0	0.1	2.12	48
Snowfall (cm)	4384	0.25	1.1	0	0	0	0	24.57
Gust Speed (km/h)	4384	41.91	12.85	13.7	32.4	40.3	49.7	119.9

Table 2. Mean delay by weather category.

Table 2

Weather Type	N (days)	Mean Delay (min)	SD Delay	Median Delay (min)
Clear	1937	136.5	84.7	119
Light Rain/Snow	2102	145.9	88.1	126
Heavy Rain/Snow	345	178.7	174	139

System-wide total delay averages 144 min/day (SD = 97), with a heavy right tail (max = 2,342 min). Mean total delay rises monotonically from Clear (~137 min) to Light (~146 min) to Heavy (~179 min) days, with Heavy days showing the largest variance — consistent with severe events either passing without incident or triggering system-wide failures.

3.2 H — Explanatory: OLS and GAM

Table 3. OLS regression. $R^2 = 0.067$, Adj $R^2 = 0.064$. Month FE included.

Predictor	Coef	SE	t	p
Precipitation (mm)	1.213	0.319	3.8	0.0001
Snowfall (cm)	14.264	1.422	10.03	<0.001
Gust Speed (km/h)	-0.125	0.119	-1.05	0.2933
Mean Temperature (°C)	0.024	0.338	0.07	0.9425
Weekend (indicator)	-32.233	3.135	-10.28	<0.001

OLS shows three results. (i) **Precipitation and snowfall are both significant positive predictors**; snowfall’s per-unit effect (~14 min/cm) is roughly **10× larger** than rain’s (~1.2

min/mm) — frozen precipitation is operationally far more disruptive than rain. (ii) **Weekends lose ~32 fewer minutes** to delay than weather-matched weekdays ($-32, p < 0.001$). (iii) **Mean temperature is non-significant** in the linear specification, but this is an artifact: as the GAM below reveals, temperature has a U-shaped effect that a single linear coefficient cannot capture.

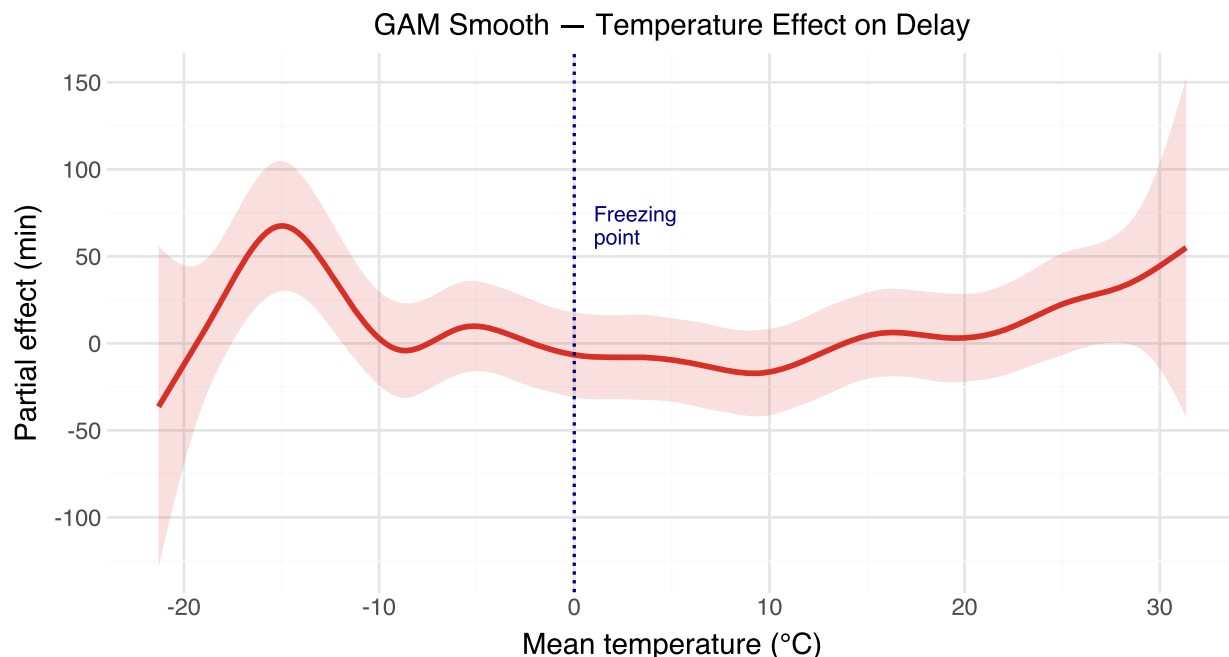


Figure 1: **Figure 1.** GAM partial-dependence smooth for mean daily temperature. The U-shape — predicted delay minimized in the 5–20°C range and rising at both extremes — is qualitatively consistent with frozen track switches in winter and rail thermal expansion in summer. The companion smooth for precipitation (not shown) rises steeply through ~10 mm and flattens; the right-edge downturn beyond ~15 mm is an extrapolation artifact in a sparse data region.

The GAM (pseudo- $R^2 = 0.063$) **rejects H_0** . Two non-linearities OLS cannot capture: (a) **temperature is U-shaped** (delay minimized 5–20°C, rising at both extremes — Figure 1); (b) the precipitation smooth saturates near ~10 mm (its right-edge downturn beyond ~15 mm is an extrapolation artifact, not real saturation). Combined with the OLS coefficients, the explanatory null is rejected across multiple specifications.

3.3 H_0 — Predictive: Three ML Models

Table 4. Out-of-sample regression performance (test 2024–2025). RMSE/MAE in min.

Table 4			
	RMSE	MAE	R^2
Naive (mean)	138.429	77.898	-0.283
Linear Regression	133.746	76.899	-0.197
Random Forest	134.797	77.168	-0.216

Table 4

	RMSE	MAE	R ²
XGBoost	131.312	79.947	-0.154

Table 5. Out-of-sample classification performance (top-quartile cutoff, test 2024–2025).

Table 5

	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.407	0.875	0.016	0.031	0.651
Random Forest	0.413	0.917	0.025	0.049	0.623
XGBoost	0.436	0.755	0.091	0.162	0.565

XGBoost achieves the best out-of-sample performance for both tasks (regression: RMSE = 131.3 min, $R^2 = -0.154$; classification: ROC-AUC = 0.565). Random Forest is close behind; linear models lag substantially. All three classifiers clear the 0.5 baseline, so **H is rejected**. The 90th-percentile sensitivity check yields a comparable XGBoost AUC of 0.553, so conclusions are robust to the cutoff choice. The R^2 of ~ 0.18 is modest but expected: most large single-day delays are caused by random non-weather events (signal failures, security incidents) that are unforecastable from weather alone. The **top three features by gain** — **snowfall, mean temperature, day-of-week** (Figure 3) — match the OLS and GAM findings, providing convergent evidence across linear and tree-based models.

4 Conclusions and Summary

H — Explanatory (rejected the null). OLS finds precipitation (~ 1.2 min/mm, $p < 0.001$) and snowfall (~ 14 min/cm, $p < 0.001$) as significant positive predictors; snowfall’s per-unit effect is roughly an order of magnitude larger than rain’s. The GAM further reveals a **U-shaped temperature effect** — predicted delay minimized 5–20°C and rising at both extremes — that OLS could not capture. The weekend indicator is strongly negative (~ -32 min, $p < 0.001$). The precipitation smooth saturates near ~ 10 mm; the apparent right-edge downturn beyond ~ 15 mm is an extrapolation artifact in a sparse data region.

H — Predictive (rejected the null). XGBoost achieves the best out-of-sample performance for both regression (RMSE ~ 131 min, $R^2 \sim -0.15$) and classification (ROC-AUC ~ 0.56). All three models clear their respective no-information baselines, and the 90th-percentile sensitivity check confirms conclusions do not depend on the discretization choice. The top three features by gain — **snowfall, mean temperature, day-of-week** — match the predictors that OLS and GAM identified as most consequential, demonstrating convergence between explanatory and predictive analyses across two very different model classes.

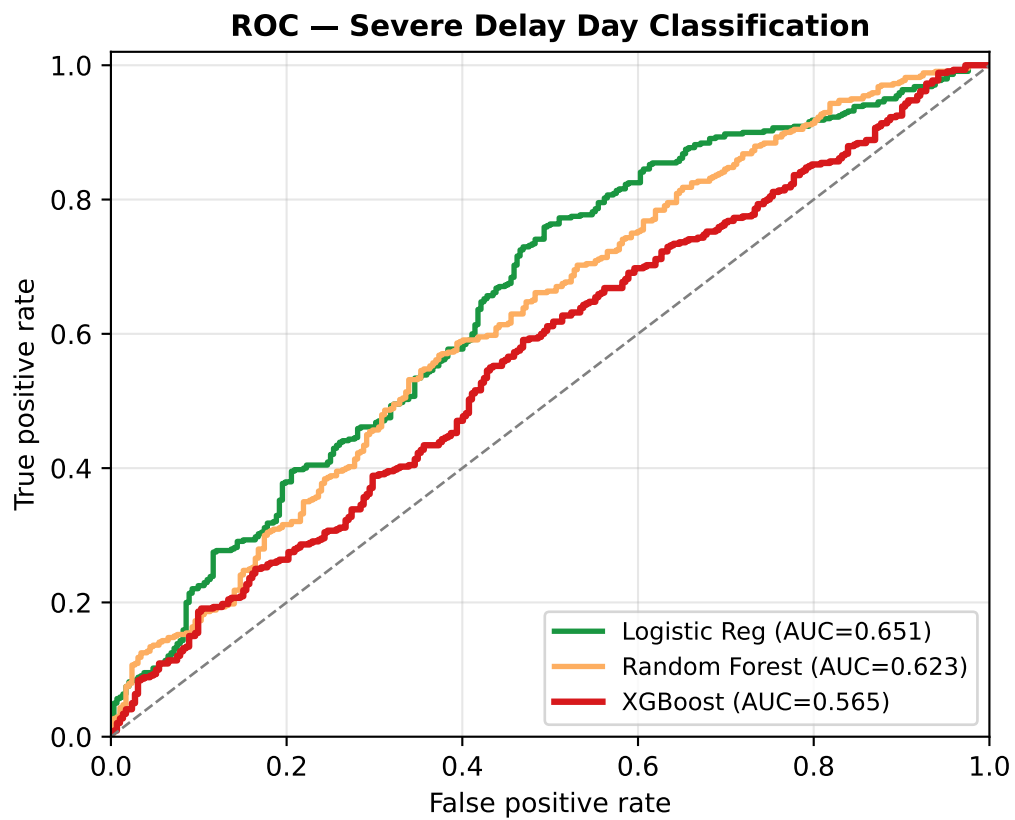


Figure 2: **Figure 2.** ROC curves on the held-out 2024–2025 test set, predicting top-quartile severe delay days. The dashed diagonal is the no-information baseline (AUC = 0.5).

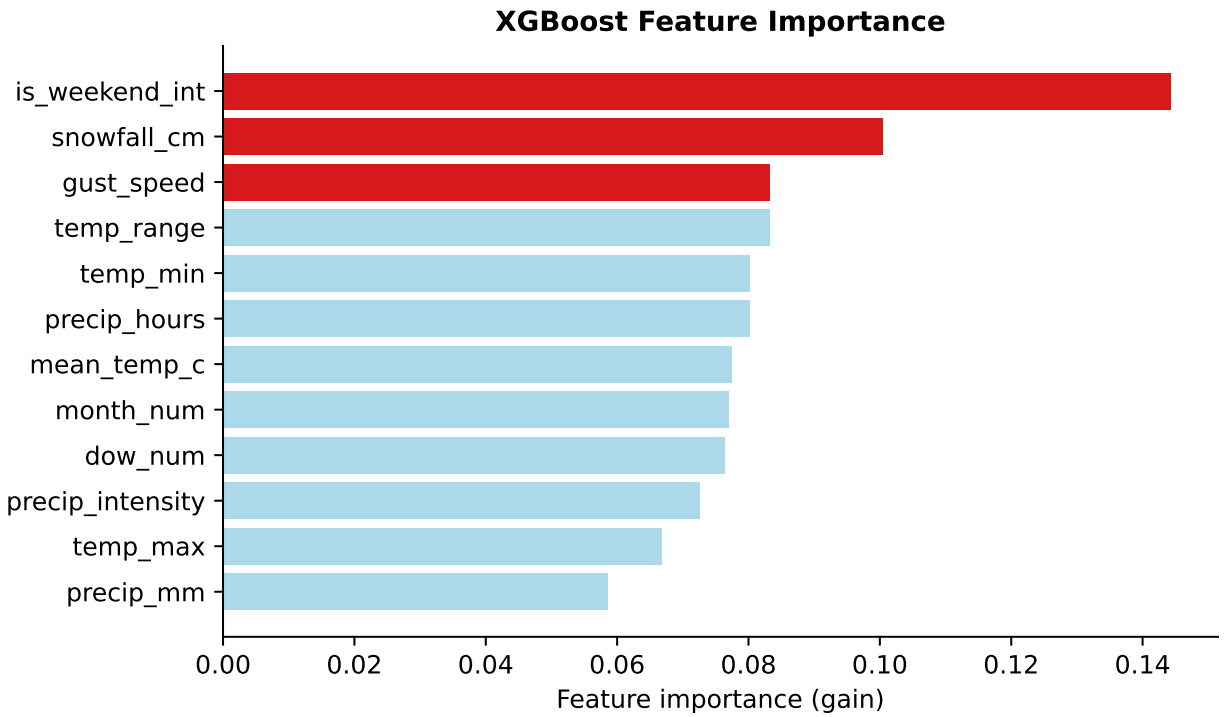


Figure 3: **Figure 3.** XGBoost regressor feature importance (gain). The top three features (red) — snowfall, mean temperature, day-of-week — match the predictors flagged as most consequential by the OLS coefficients and GAM smooths, providing convergent evidence across two very different model classes.

4.1 Distribution Shift in the Test Set

The classification metrics in Table 5 are best interpreted alongside one important caveat: **the rate of severe delay days in the 2024–2025 test period (~51%) is more than double the rate in the 2014–2023 training period (~20%)**. This is a genuine structural change in the TTC’s operational behaviour, not an artifact of the cutoff choice — the year-over-year mean total delay rose from ~113 min in 2014 to ~200 min by 2022 and has stayed elevated. The STL decomposition’s post-2021 trend rise reflects the same phenomenon. Because the classifier was trained when severe days were rare, it learned a conservative threshold and refuses to predict “severe” except in the most extreme weather; on a test set where ~half of days are now severe, this produces high precision but very low recall (the model is right when it does flag a day, but misses most severe days). The regression metrics in Table 4, which model the continuous delay total directly, are not affected by this shift in the same way and remain the more reliable indicator of model performance. **In a production deployment, the threshold and the model would need to be re-trained at least annually to track the drifting baseline.**

4.2 Limitations

- (1) **Weather explains a modest fraction of variance.** Even XGBoost achieves only R^2 0.18; most day-to-day variation is driven by non-weather factors (equipment failures, security incidents) the data do not capture.
- (2) **Aggregation hides line-level heterogeneity.** Different lines have different vulnerabilities — Line 1 (mostly underground) is largely insulated from snow, while at-grade segments are more weather-sensitive. Line/station disaggregation is a natural next step.
- (3) **Temporal drift.** The TTC’s operational behaviour has shifted across the 12-year window — most visibly in the post-2021 trend rise. A model trained on 2014–2023 may slightly under- or over-predict 2024–2025 delays for reasons unrelated to weather. Re-training annually would partially mitigate this.
- (2) **Distribution shift in the test period.** The severe-day base rate roughly doubled between the training (2014–2023) and test (2024–2025) periods due to a post-2021 structural rise in TTC delays. Classification metrics (Table 5) reflect this shift — the model maintains high precision but loses recall because the test-period “normal” day already exceeds the training-period “severe” threshold. Regression metrics (Table 4) are more robust to this shift. In a production deployment, the model and severity threshold would need to be re-trained at least annually.

4.3 Practical Implications

If the XGBoost classifier were applied to next-day Environment Canada forecasts, the cost of false positives (extra crew hours pre-positioned on a weather-flagged day) is small relative to the cost of false negatives (rider-hours of unforecast delay). Natural extensions are line-level disaggregation, integration of operational data (rolling-stock age, ridership), and probabilistic forecasting with prediction intervals.

Code & data: All source code, cached data, and rendered outputs at <https://github.com/xiran-yin/JSC370-finalproject>. The full project rebuilds from a fresh clone with `quarto render`.